



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 9
по курсу «Методы оптимизации»
«Метод Ньютона»

ИУ9-81Б Егорова А. А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Метод Ньютона. Сравнить с модификацией, когда обратный гессиан вычисляется один раз.

2 Результаты

```
1 using LinearAlgebra
2 using Plots
3 plotly()
4 function modified_newton(f, grad_f, hess_f, x0; eps=1e-6, max_iter=1000)
5     x = copy(x0)
6     H_inv = inv(hess_f(x))
7     history = [copy(x)]
8     grad_norms = [norm(grad_f(x))]
9
10    for k in 1:max_iter
11        g = grad_f(x)
12        if norm(g) < eps
13            break
14        end
15        d = -H_inv * g
16        function phi(a)
17            return f(x + a * d)
18        end
19        a_opt = golden_section_search(phi, 0.0, 2.0)
20        x = x + a_opt * d
21        push!(history, copy(x))
22        push!(grad_norms, norm(g))
23    end
24    return x, history, grad_norms
25 end
26 function newton_method(f, grad_f, hess_f, x0; eps=1e-6, max_iter=1000)
27     x = copy(x0)
28     history = [copy(x)]
29
30    for k in 1:max_iter
31        g = grad_f(x)
32        if norm(g) < eps
33            break
34        end
35        H = hess_f(x)
```

```

36         H_inv = inv(H)
37         d = -H_inv * g
38         function phi(a)
39             return f(x + a * d)
40         end
41         a_opt = golden_section_search(phi, 0.0, 2.0)
42         x = x + a_opt * d
43         push!(history, copy(x))
44     end
45     return x, history
46 end
47
48 function golden_section_search(phi, a=0.0, b=2.0; tol=1e-6, max_iter
    =100)
49     phi_gold = (sqrt(5) - 1) / 2
50     c = b - phi_gold * (b - a)
51     d = a + phi_gold * (b - a)
52     fc = phi(c)
53     fd = phi(d)
54
55     for i in 1:max_iter
56         if fc < fd
57             b = d
58             d = c
59             fd = fc
60             c = b - phi_gold * (b - a)
61             fc = phi(c)
62         else
63             a = c
64             c = d
65             fc = fd
66             d = a + phi_gold * (b - a)
67             fd = phi(d)
68         end
69         if abs(b - a) < tol
70             break
71         end
72     end
73     return (a + b) / 2
74 end
75
76 function rosenbrock(x)
77     return (1 - x[1])^2 + 100 * (x[2] - x[1]^2)^2
78 end
79
80 function schwefel(x)

```

```

81     return 418.9829 * 2 - x[1] * sin(sqrt(abs(x[1]))) - x[2] * sin(sqrt(
    abs(x[2])))
82 end
83
84 function rastrigin(x)
85     return (x[1]^2 - 10 * cos(2 * x[1]) + 10) + (x[2]^2 - 10 * cos(2
    * x[2]) + 10)
86 end
87
88 function rosenbrock_grad(x)
89     return [-2.0 * (1.0 - x[1]) - 400.0 * x[1] * (x[2] - x[1]^2),
90             200.0 * (x[2] - x[1]^2)]
91 end
92
93 function rosenbrock_hess(x)
94     return [2.0 - 400.0 * (x[2] - 3.0 * x[1]^2)    -400.0 * x[1];
95             -400.0 * x[1]                            200.0]
96 end
97
98 function rastrigin_grad(x)
99     n = length(x)
100    A = 10.0
101    return [2.0 * x[i] + 2.0 * pi * A * sin(2.0 * pi * x[i]) for i in 1:
    n]
102 end
103
104 function rastrigin_hess(x)
105     n = length(x)
106     A = 10.0
107     H = zeros(n, n)
108     for i in 1:n
109         H[i, i] = 2.0 + 4.0 * pi^2 * A * cos(2.0 * pi * x[i])
110     end
111     return H
112 end
113
114 function schwefel_grad(x)
115     n = length(x)
116     grad = zeros(n)
117     for i in 1:n
118         s = sqrt(abs(x[i]))
119         grad[i] = -sin(s) - (x[i] * cos(s)) / (2.0 * s)
120         if x[i] < 0
121             grad[i] = -grad[i]
122         end
123     end

```

```

124     return grad
125 end
126
127 function schwefel_hess(x)
128     n = length(x)
129     H = zeros(n, n)
130     for i in 1:n
131         s = sqrt(abs(x[i]))
132         if abs(x[i]) > 1e-10
133             if x[i] > 0
134                 H[i, i] = -(cos(s) / (2.0 * s)) - (cos(s) / (2.0 * s)) +
(x[i] * sin(s)) / (4.0 * x[i])
135             else
136                 H[i, i] = (cos(s) / (2.0 * s)) + (cos(s) / (2.0 * s)) -
(x[i] * sin(s)) / (4.0 * x[i])
137             end
138         else
139             H[i, i] = 0.0
140         end
141     end
142     return H
143 end
144
145 function plot_3d_trajectory(f, history_mod, history_std, x_range,
y_range, title)
146     x_vals_mod = [h[1] for h in history_mod]
147     y_vals_mod = [h[2] for h in history_mod]
148     z_vals_mod = [f([x_vals_mod[i], y_vals_mod[i]]) for i in 1:length(
x_vals_mod)]
149
150     x_vals_std = [h[1] for h in history_std]
151     y_vals_std = [h[2] for h in history_std]
152     z_vals_std = [f([x_vals_std[i], y_vals_std[i]]) for i in 1:length(
x_vals_std)]
153
154     X = range(x_range[1], x_range[2], length=100)
155     Y = range(y_range[1], y_range[2], length=100)
156     Z = [f([x, y]) for x in X, y in Y]
157
158     p = surface(X, Y, Z, alpha=0.7, c=:viridis, title=title, size=(800,
800),
159                 camera=(30, 30), label="Surface")
160     plot!(p, x_vals_mod, y_vals_mod, z_vals_mod, linewidth=2, color=:red
,
161           marker=:circle, markersize=3, label="Modified Newton")

```

```

162     plot!(p, x_vals_std, y_vals_std, z_vals_std, linewidth=2, color=:
blue,
163         marker=:square, markersize=3, label="Standard Newton")
164
165     return p
166 end
167
168 println("Rosenbrock function")
169 x0_rosen = [-1.5, 1.27]
170
171 time_mod_rosen = @elapsed begin
172     x_opt_mod_rosen, history_mod_rosen, _ = modified_newton(rosenbrock,
rosenbrock_grad, rosenbrock_hess, x0_rosen, eps=1e-6)
173 end
174
175 time_std_rosen = @elapsed begin
176     x_opt_std_rosen, history_std_rosen = newton_method(rosenbrock,
rosenbrock_grad, rosenbrock_hess, x0_rosen, eps=1e-6)
177 end
178
179 println("Modified Newton iterations: ", length(history_mod_rosen) - 1)
180 println("Modified Newton time: $(round(time_mod_rosen, digits=6)) s")
181 println("Standard Newton iterations: ", length(history_std_rosen) - 1)
182 println("Standard Newton time: $(round(time_std_rosen, digits=6)) s")
183 println("Time difference: $(round(time_std_rosen - time_mod_rosen,
digits=6)) s")
184 println()
185
186 println("Rastrigin function")
187 x0_rast = [3.1, 4.1]
188
189 time_mod_rast = @elapsed begin
190     x_opt_mod_rast, history_mod_rast, _ = modified_newton(rastrigin,
rastrigin_grad, rastrigin_hess, x0_rast, eps=1e-6)
191 end
192
193 time_std_rast = @elapsed begin
194     x_opt_std_rast, history_std_rast = newton_method(rastrigin,
rastrigin_grad, rastrigin_hess, x0_rast, eps=1e-6)
195 end
196
197 println("Modified Newton iterations: ", length(history_mod_rast) - 1)
198 println("Modified Newton time: $(round(time_mod_rast, digits=6)) s")
199 println("Standard Newton iterations: ", length(history_std_rast) - 1)
200 println("Standard Newton time: $(round(time_std_rast, digits=6)) s")

```

```

201 println("Time difference: $(round(time_std_rast - time_mod_rast, digits
    =6)) s")
202 println()
203
204 println("Schwefel function")
205 x0_schwefel = [400.0, 400.0]
206
207 time_mod_schwefel = @elapsed begin
208     x_opt_mod_schwefel, history_mod_schwefel, _ = modified_newton(
        schwefel, schwefel_grad, schwefel_hess, x0_schwefel, eps=1e-6)
209 end
210
211 time_std_schwefel = @elapsed begin
212     x_opt_std_schwefel, history_std_schwefel = newton_method(schwefel,
        schwefel_grad, schwefel_hess, x0_schwefel, eps=1e-6)
213 end
214 println("Modified Newton iterations: ", length(history_mod_schwefel) -
    1)
215 println("Modified Newton time: $(round(time_mod_schwefel, digits=6)) s")
216 println("Standard Newton iterations: ", length(history_std_schwefel) -
    1)
217 println("Standard Newton time: $(round(time_std_schwefel, digits=6)) s")
218 println("Time difference: $(round(time_std_schwefel - time_mod_schwefel,
    digits=6)) s")
219 println()
220
221 p1 = plot_3d_trajectory(rosenbrock, history_mod_rosen, history_std_rosen
    ,
222     [-2.0, 2.0], [-1.0, 3.0], "Rosenbrock Function")
223 p2 = plot_3d_trajectory(rastrigin, history_mod_rast, history_std_rast,
224     [-5.0, 5.0], [-5.0, 5.0], "Rastrigin Function")
225 p3 = plot_3d_trajectory(schwefel, history_mod_schwefel,
    history_std_schwefel,
226     [-500.0, 500.0], [-500.0, 500.0], "Schwefel
    Function")
227
228 gui(p1)
229 gui(p2)
230 gui(p3)

```

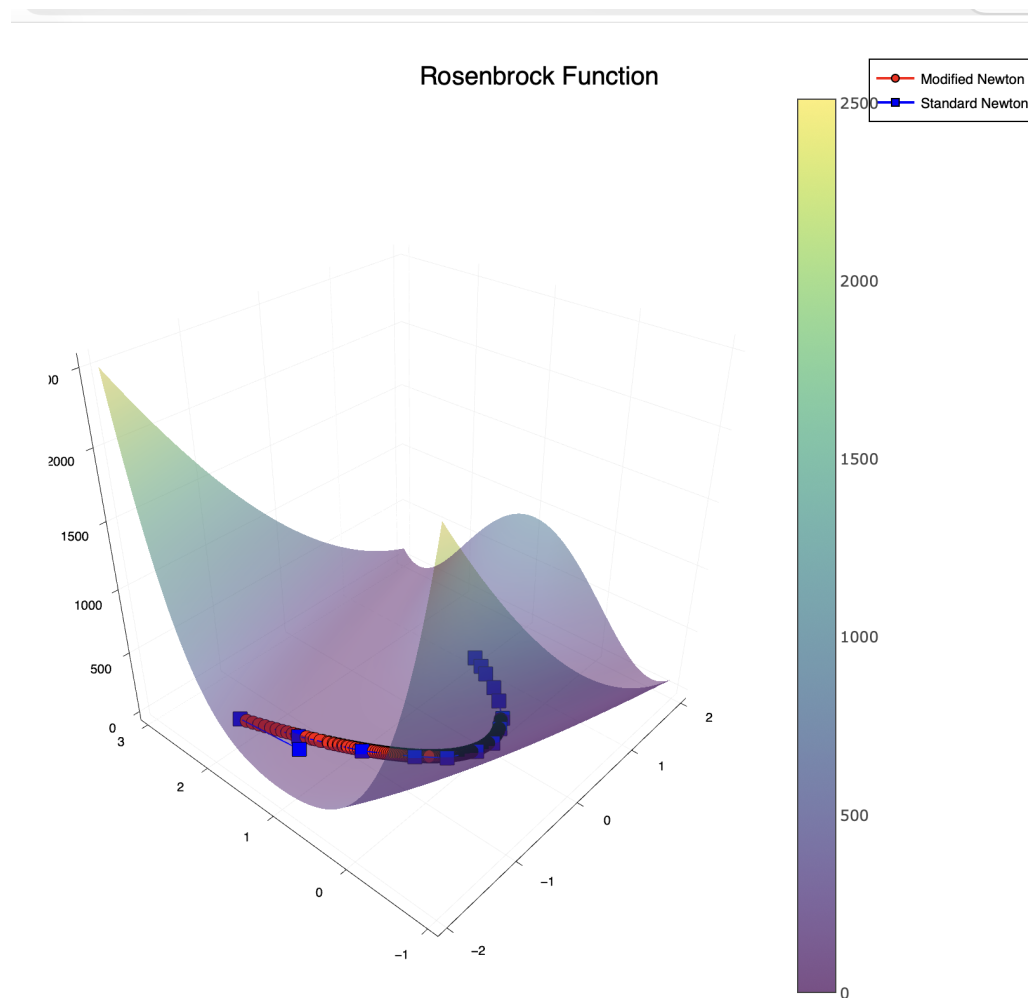


Рис. 1 — Результат

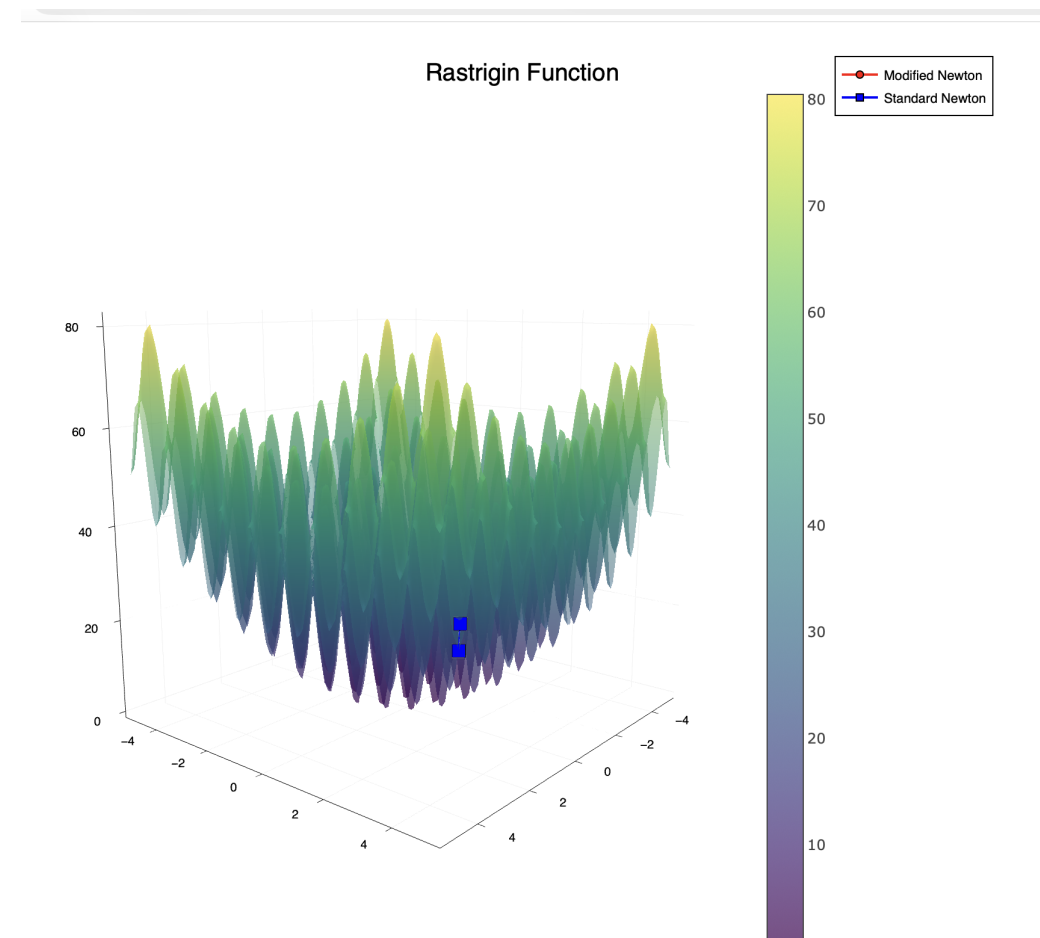


Рис. 2 — Результат

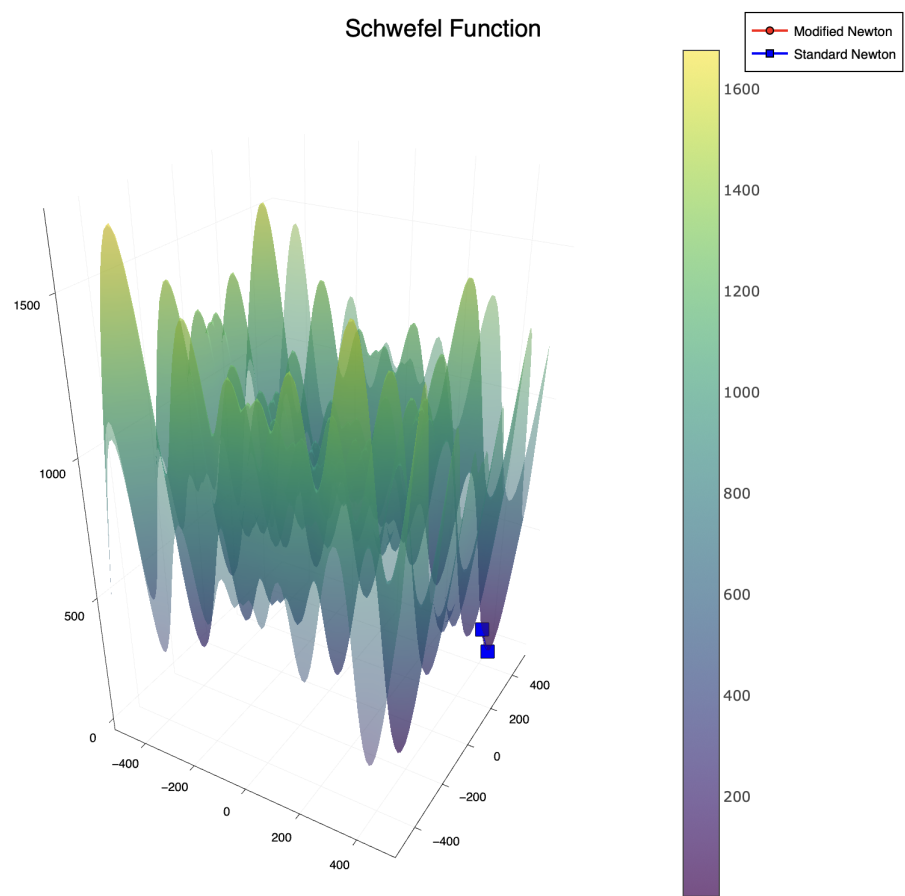


Рис. 3 — Результат

```
Rosenbrock function
Modified Newton iterations: 1000
Modified Newton time: 0.037186 s
Standard Newton iterations: 17
Standard Newton time: 0.016101 s
Time difference: -0.021085 s

Rastrigin function
Modified Newton iterations: 4
Modified Newton time: 0.037136 s
Standard Newton iterations: 3
Standard Newton time: 0.013696 s
Time difference: -0.02344 s

Schwefel function
Modified Newton iterations: 1
Modified Newton time: 0.031325 s
Standard Newton iterations: 1
Standard Newton time: 0.01344 s
Time difference: -0.017885 s
```

Рис. 4 — Результат